

Technical Architecture Document: Vibe Problem Solver

Overview

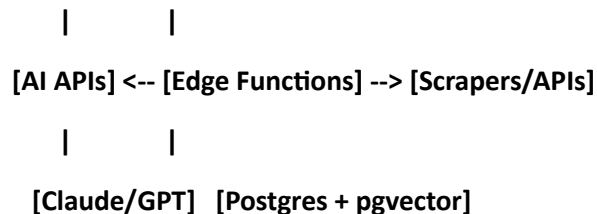
This document outlines the system architecture for the Vibe Problem Solver MVP—a field-first web app using Next.js (frontend), Supabase (backend/database/auth), and Claude/GPT for AI generation. The design prioritizes scalability, security, and rapid iteration with a clear separation: Next.js handles UI/routing/server actions, Supabase manages data/auth/realtime, and edge functions process async jobs like problem scraping.

Architecture follows a serverless-first approach: Vercel for hosting, Supabase Postgres for relational data + vector extensions for embeddings, and LLM orchestration via API routes. Total MVP build assumes solo dev with 10-week timeline from PRD.

High-Level System Diagram

text

[Users] --> [Vercel/Next.js App] --> [Supabase Project]



- **Data Flow:** User selects domain → Server action triggers discovery → Ranks problems → Generates solutions/roadmaps → Stores in Supabase → Serves via realtime subscriptions.

Tech Stack

Layer	Primary	Alternatives	Rationale
Frontend	Next.js 15 (App Router), Tailwind CSS, Framer Motion	Remix	Server Components for data fetching, ISR for feeds

Layer	Primary	Alternatives	Rationale
Backend/DB	Supabase (Postgres + pgvector), Prisma ORM	Convex	Built-in auth, realtime, edge functions; scales to 1K DAU
Auth	Clerk (MFA, roles) + Supabase fallback	Supabase Auth	Subscription tiers, OAuth (Google/GitHub)
AI/ML	Claude 3.5 Sonnet (primary), GPT-5o	Blackbox AI	Reasoning for roadmaps; cache responses in Supabase
Monitoring	Sentry, Vercel Analytics	PostHog	Error tracking, performance
Deployment	Vercel (FE), Supabase (BE)	Netlify/Railway	Git-based CI/CD, auto-scaling

Data Model (Postgres Schemas)

sql

-- Core tables via Prisma schema

model User {

id String @id @default(cuid())

email String @unique

domains String[] // JSON array: ["education", "finance"]

savedProblems String[] // ProblemCluster IDs

```
roadmaps    Roadmap[]

subscription String @default("free") // free/pro/enterprise
}
```

```
model ProblemCluster {

  id        String @id @default(cuid())

  canonicalText String

  painIndex  Float

  difficulty String // easy/medium/hard

  domainTags String[]

  sources    Json // [{url, authority}]

  solutions  Solution[]

  createdAt  DateTime @default(now())

}
```

```
model Solution {

  id        String @id @default(cuid())

  problemId String

  headline  String

  layers    Json // 6-layer structure
```

```

techStack  Json      // {fe, be, db}

problem    ProblemCluster @relation(fields: [problemId], references: [id])

roadmaps   Roadmap[]

}

```

```

model Roadmap {
  id          String @id @default(cuid())
  solutionId  String
  phases      Json[] // [{name, tasks, duration}]
  teamSize    String // solo/small/large
  documents   Document[]
  solution    Solution @relation(fields: [solutionId], references: [id])
}

```

- Vector Extension: Add pgvector for semantic clustering: ALTER TABLE ProblemCluster ADD COLUMN embedding vector(1536);
- RLS Policies: ENABLE ROW LEVEL SECURITY; on all tables; e.g., CREATE POLICY user_problems ON ProblemCluster FOR SELECT USING (auth.uid() = ANY(domains));

API Endpoints (OpenAPI Stub)

text

openapi: 3.0.0

paths:

/api/problems:

get:

summary: Fetch ranked problems by domain

parameters:

- name: domain

in: query

schema: {type: string}

- name: limit

in: query

schema: {type: integer, default: 20}

responses:

200:

content:

application/json:

schema:

type: array

items: {\$ref: '#/components/schemas/ProblemCard'}

/api/[problemId]/solution:

post:

summary: Generate layered solution

requestBody:

content:

application/json:

schema: {type: object}

responses:

200:

content:

application/json:

schema: {\$ref: '#/components/schemas/Solution'}

/api/roadmap:

post:

summary: Create phased roadmap

requestBody:

required: true

content:

application/json:

schema: {properties: {solutionId: {type: string}, teamSize: {type: string}}}

responses:

201:

content:

application/json:

schema: {\$ref: '#/components/schemas/Roadmap'}

Server Actions Example (Next.js /app/actions.ts):

ts

'use server'

import { createClient } from '@utils/supabase/server'

import { callClaude } from '@lib/ai'

export async function generateRoadmap(solutionId: string, teamSize: string) {

const supabase = createClient()

// Fetch solution → LLM prompt → Insert roadmap

const roadmap = await callClaude({ solutionId, teamSize })

const { data, error } = await supabase.from('roadmaps').insert(roadmap)

if (error) throw error

return data

}

Rate-limited via Clerk middleware: 10/min per user.

Backend Processing Pipeline

1. Ingestion (Edge Function): Cron job scrapes Reddit/HN via APIs → Extracts problems → Computes Pain Index.
2. Clustering: Embed texts → pgvector cosine similarity → Merge duplicates (threshold 0.85).
3. AI Layer: /api/generate → Claude prompt template → Cache in Redis/Supabase.
4. Realtime: Supabase subscriptions for feed updates: `supabase.channel('problems').on('postgres_changes', ...)`

Pain Index Cron (Supabase Edge Function, Deno/TS):

ts

// Deno script: recalculate Pi hourly

```
const { data } = await supabase.from('raw_mentions').select('*').gte('age_hours', 0).lte('age_hours', 168)
for (const mention of data) {
  const pi = (mention.volume * mention.sentiment) / mention.age * mention.authority
  await supabase.from('problem_clusters').upsert({ id: mention.cluster_id, pain_index: pi })
}
```

Security & Observability

- Rate Limits: Upstash Redis or Cloudflare: /solve: 10/min, /scrape: 50/hr.
- Input Validation: Zod schemas on all actions; never trust client inputs.
- Secrets: Vercel env vars; no API keys in client bundle.
- Monitoring: Sentry for errors, Supabase Logs for DB, Vercel Speed Insights.
- Compliance: RLS + GDPR retention (problems >90d auto-archive).

Deployment & CI/CD

1. git push → Vercel builds Next.js (auto deploys preview).

2. Supabase migrations: prisma migrate deploy.
3. Env: SUPABASE_URL, SUPABASE_ANON_KEY, CLAUDE_API_KEY.
4. Scaling: Vercel auto-scales FE; Supabase Compute Add-ons for >1K DAU.

Risks & Mitigations

Risk	Impact	Mitigation
LLM Cost	High	Cache 80% responses; fallback to templates
Scraper Bans	Med	Proxy rotation + polite backoffs
DB Hotspots	Low	Indexes on domain_tags, pain_index; read replicas

Next Steps: Implement Prisma schema migration (Day 1), POC edge function for Pi calc (Day 3), review by Feb 7. Export as PDF/MD via remark CLI.